

Probability and Statistics

Coursework

December 16, 2021

Question 1

(a)

We have that:

$$\begin{aligned} \mathbb{E}[\bar{W}] &= \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n \frac{g(X_i)}{f(X_i)}\right] = \frac{1}{n} \mathbb{E}\left[\sum_{i=1}^n \frac{g(X_i)}{f(X_i)}\right] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}\left[\frac{g(X_i)}{f(X_i)}\right] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}\left[\frac{g(X)}{f(X)}\right] = \frac{1}{n} n \mathbb{E}\left[\frac{g(X)}{f(X)}\right] \\ &= \mathbb{E}\left[\frac{g(X)}{f(X)}\right] = \int_{-\infty}^{+\infty} \frac{g(x)}{f(x)} f(x) dx = \int_{-\infty}^{+\infty} g(x) dx = I \end{aligned}$$

which implies that the sample mean of the weights is an unbiased estimator of the integral I .

(b)

Notice that:

$$\text{Var}\left[\frac{g(X)}{f(X)}\right] = \mathbb{E}\left[\left(\frac{g(X)}{f(X)}\right)^2\right] - \mathbb{E}\left[\frac{g(X)}{f(X)}\right]^2 = \mathbb{E}\left[\left(\frac{g(X)}{f(X)}\right)^2\right] - I^2$$

Since $(X_i)_{i \in \overline{1, n}}$ are i.i.d. we have that

$$\begin{aligned} \text{Var}[\bar{W}] &= \text{Var}\left[\frac{1}{n} \sum_{i=1}^n \frac{g(X_i)}{f(X_i)}\right] = \frac{1}{n^2} \text{Var}\left[\sum_{i=1}^n \frac{g(X_i)}{f(X_i)}\right] = \frac{1}{n^2} \sum_{i=1}^n \text{Var}\left[\frac{g(X_i)}{f(X_i)}\right] = \frac{1}{n^2} \sum_{i=1}^n \text{Var}\left[\frac{g(X)}{f(X)}\right] \\ &= \frac{1}{n^2} n \text{Var}\left[\frac{g(X)}{f(X)}\right] = \frac{1}{n} \text{Var}\left[\frac{g(X)}{f(X)}\right] = \frac{1}{n} \left(\mathbb{E}\left[\left(\frac{g(X)}{f(X)}\right)^2\right] - I^2\right) \end{aligned}$$

which implies that the variance of the estimator $\bar{W}$ is proportional to $\left(\mathbb{E}\left[\left(\frac{g(X)}{f(X)}\right)^2\right] - I^2\right)$.

The standard deviation of $\bar{W}$ is given by :

$$\sigma_{\bar{W}} = \sqrt{\text{Var}[\bar{W}]} = \frac{1}{\sqrt{n}} \cdot \sqrt{\mathbb{E}\left[\left(\frac{g(X)}{f(X)}\right)^2\right] - I^2}$$

We have 2 cases:

Case 1: $E \left[\left(\frac{g(X)}{f(X)} \right)^2 \right] - I^2 = 0$

Then the standard deviation $\sigma_{\overline{W}}$ is equal to 0 regardless of the value of n .

Case 2: $E \left[\left(\frac{g(X)}{f(X)} \right)^2 \right] - I^2 > 0$

Then the standard deviation of the estimator $\overline{W}$ scales proportional to $\frac{1}{\sqrt{n}} = n^{-\frac{1}{2}}$.

(c)

Suppose that $f(x) \propto g(x)$. That is, there is a constant $c \in \mathbb{R}$ such that $g(x) = cf(x)$.

Then we have that:

$$\int_{-\infty}^{+\infty} g(x) dx = I \Rightarrow \int_{-\infty}^{+\infty} cf(x) dx = I \Rightarrow c \int_{-\infty}^{+\infty} f(x) dx = I \Rightarrow c = I \quad (\text{since } X \text{ has pdf } f(x))$$

which implies that

$$\text{Var} [\overline{W}] = \frac{1}{n} \left(E \left[\left(\frac{g(X)}{f(X)} \right)^2 \right] - I^2 \right) = \frac{1}{n} (E [c^2] - I^2) = \frac{1}{n} (c^2 - I^2) = \frac{1}{n} (I^2 - I^2) = 0$$

Therefore the variance of the estimator is 0, which implies that $f(x) \propto g(x)$ is the ideal choice for the RV X . But this choice is impractical to use since the constant of proportionality needed is I itself. (which is the quantity that we want to estimate)

(d)

In order to compute the **estimate** using samples from $N(0, \sigma^2)$, we use the following algorithm:

Step 1: Generate a sample of size $n = 20$ from $N(0, 1)$.

This can be done using `numpy.random.randn()`.

Step 2: Multiply every value by σ . Then we get a sample of size $n = 20$ from $N(0, \sigma^2)$.

Step 3: Apply the function $x \rightarrow \frac{g(x)}{\text{pdf of } N(0, \sigma^2)}$ to every element of the sample.

In this way, we obtain the weights needed to estimate the integral.

Step 4: Compute the sample mean of the weights. Then this will be an estimation of the integral.

This can be done by using `numpy.mean()`.

In order to compute the **confidence interval** for our estimation, we need to use a Student's t -distribution, as we are assuming that the weights are normally distributed (we know the mean of the normal distribution = the mean of the weights, but we don't know the standard deviation).

By computing the standard error mean, we can use the formula for a $100(1 - \alpha)\%$ confidence interval for I in order to get the desired result.

Alternatively, this can be done by using `scipy.stats.t.interval()`.

In the given case, we obtain the following result:

estimate of the integral: 1.6936228751468119

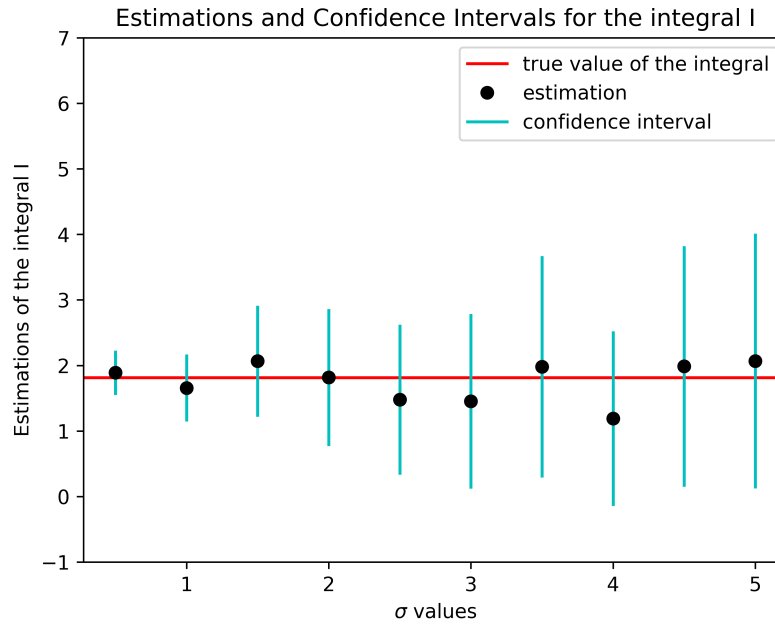
95% confidence interval: (1.1776665653117973, 2.2095791849818265)

(e)

The list of values of σ is [0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0].

For every σ in the given list, we apply the previous 2 algorithms in order to get the estimation and confidence interval. Then we plot those in figure with σ on the x -axis and I on the y -axis.

Moreover, we also add a horizontal line to the plot at the true value of $I = 1.81280$.

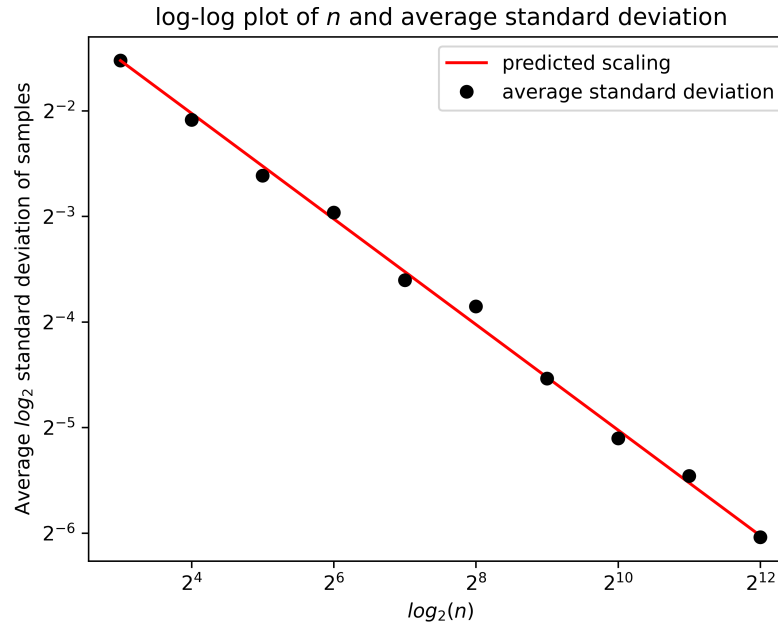


(f)

For each n in $[2^3, 2^4, \dots, 2^{12}]$, we repeat the experiments 100 times and get 100 estimations of the integral. Then we compute the standard deviation of these estimations by using `numpy.std()`.

At part (b), we predicted that the standard deviation scales as $n^{-\frac{1}{2}}$. We then plot the line $y = \sigma_{n=8} \left(\frac{n}{8}\right)^\lambda$, where $\sigma_{n=8}$ is the standard deviation of the 100 estimates for the $n = 8$ case and $\lambda = -\frac{1}{2}$.

The log-log plot was constructed using `matplotlib.pyplot.loglog()`.



(g)

The standard deviation of the random variable X is given by:

$$\sigma = \sigma_X = \sqrt{\mathbb{E} \left[\left(\frac{g(X)}{f(X)} \right)^2 \right] - I^2}$$

The standard deviation of the estimator is given by

$$\sigma_{\bar{W}} = \frac{1}{\sqrt{n}} \cdot \sqrt{\mathbb{E} \left[\left(\frac{g(X)}{f(X)} \right)^2 \right] - I^2} = \frac{1}{\sqrt{n}} \cdot \sigma_X$$

In order for the estimator to be a good one, it needs to have a low standard deviation (otherwise it can give us values which are far from the true value of the integral I). If the pdf $f(x)$ is much “narrower” or “broader” than $g(x)$, then the standard deviation of the random variable X is a large number. Thus, if we want to estimate $g(x)$ using $f(x)$, we will need a large number of samples.

But this makes the algorithm impractical to use, since for a given case, we don't know how many samples we need in order for the estimator to have a low standard deviation.

Question 2

For every $x \in \mathbb{R}$, we have that:

$$\begin{aligned} F_X(x) &= P(X \leq x) = P(F^{-1}(U) \leq x) = P(U \leq F(x)) && \text{(since } F \text{ is increasing)} \\ &= F(x) && \text{(since } \forall y \in \mathbb{R}, 0 \leq F(y) \leq 1 \text{ and } U \sim U(0, 1)) \end{aligned}$$

which implies that the cdf of X is $F(x)$.

Question 3

(a)

Since $f_X(x)$ is a pdf, we have that $\int_{-\infty}^{\infty} f_X(x) dx = 1$.

$$\begin{aligned} \int_{-\infty}^{\infty} f_X(x) dx &= 1 \Rightarrow \int_{x_{\min}}^{\infty} c \cdot \left(\frac{x}{x_{\min}} \right)^{-\lambda} dx = 1 && \text{(by the definition of } f_X(x)) \\ &\Rightarrow c \cdot \frac{1}{x_{\min}^{-\lambda}} \cdot \left[\frac{x^{-\lambda+1}}{-\lambda+1} \right]_{x=x_{\min}}^{x=\infty} = 1 \Rightarrow c \cdot \frac{1}{x_{\min}^{-\lambda}} \cdot \left[0 - \frac{x_{\min}^{-\lambda+1}}{-\lambda+1} \right] = 1 \\ &\Rightarrow c \cdot \frac{x_{\min}}{-\lambda+1} = -1 \Rightarrow c = \frac{\lambda-1}{x_{\min}} \end{aligned}$$

Thus $c = \frac{\lambda-1}{x_{\min}}$.

(b)

Let $\underline{x} = (x_1, \dots, x_n)$ be the measured masses of the n galaxies.

The log-likelihood is given by:

$$\ell(\lambda, \underline{x}) = \sum_{i=1}^n \log[f_{X|\lambda}(x_i)] = \sum_{i=1}^n \log[(\lambda-1) \cdot x_i^{-\lambda}] = n \cdot \log(\lambda-1) - \lambda \cdot \sum_{i=1}^n \log(x_i)$$

For every $i \in \overline{1, n}$, we have that $x_i \geq 1$, which implies that $\sum_{i=1}^n \log(x_i) \geq 0$.

Case 1: $\sum_{i=1}^n \log(x_i) > 0$

We differentiate $\ell(\lambda, \underline{x})$:

$$\frac{\partial}{\partial \lambda} \ell(\lambda, \underline{x}) = \frac{n}{\lambda-1} - \sum_{i=1}^n \log(x_i)$$

Setting this derivative equal to 0, we get:

$$\frac{n}{\hat{\lambda}-1} - \sum_{i=1}^n \log(x_i) = 0 \Rightarrow \hat{\lambda} = 1 + \frac{n}{\sum_{i=1}^n \log(x_i)}$$

Now we find the second derivate of $\ell(\lambda, \underline{x})$:

$$\frac{\partial^2}{\partial^2 \lambda} \ell(\lambda, \underline{x}) = \frac{-n}{(\lambda - 1)^2}$$

which is < 0 for every $\lambda > 1$.

Therefore $\hat{\lambda} = 1 + \frac{n}{\sum_{i=1}^n \log(x_i)}$ is the MLE for λ .

Case 2: $\sum_{i=1}^n \log(x_i) = 0$

Then the log-likelihood is given by:

$$\ell(\lambda, \underline{x}) = n \cdot \log(\lambda - 1)$$

In this case we don't have a MLE, since this function is defined on $(0, \infty)$ and is strictly increasing.

(c)

The pdf of the power law distribution is $f_X(x) = \begin{cases} (\lambda - 1)x^{-\lambda} & , \text{ if } x \geq 1 \\ 0 & , \text{ if } x < 1 \end{cases}$

For any $x < 1$, we have that $F_X(x) = 0$.

For any $x \geq 1$, we have that:

$$F_X(x) = \int_{-\infty}^x f_X(t) dt = \int_1^x f_X(t) dt = \int_1^x (\lambda - 1)t^{-\lambda} dt = (\lambda - 1) \left[\frac{t^{-\lambda+1}}{-\lambda+1} \right]_{t=1}^{t=x} = 1 - x^{-\lambda+1}$$

Notice that $F_X(\mathbb{R}) = [0, 1]$.

[since $F_X(x) = 0$ for every $x \leq 1$, F_X is continuous and $\lim_{x \rightarrow \infty} F_X(x) = 1$]

For every $y \in (0, 1)$, we have that $F_X^{-1}(\{y\}) = \{(1 - y)^{\frac{1}{-\lambda+1}}\}$.

[since $1 - x^{-\lambda+1} = y \Leftrightarrow x = (1 - y)^{\frac{1}{-\lambda+1}}$]

For $y = 0$, we have that $F_X^{-1}(\{0\}) = (-\infty, 1]$.

Therefore the quartile function $Q : (0, 1) \rightarrow (1, \infty)$ is given by

$$Q(y) = (1 - y)^{\frac{1}{1-\lambda}} , \text{ for every } y \in (0, 1)$$

(d)

In order to generate this plot, I used the following algorithm:

Step 1: Generate n samples of $U(0, 1)$.

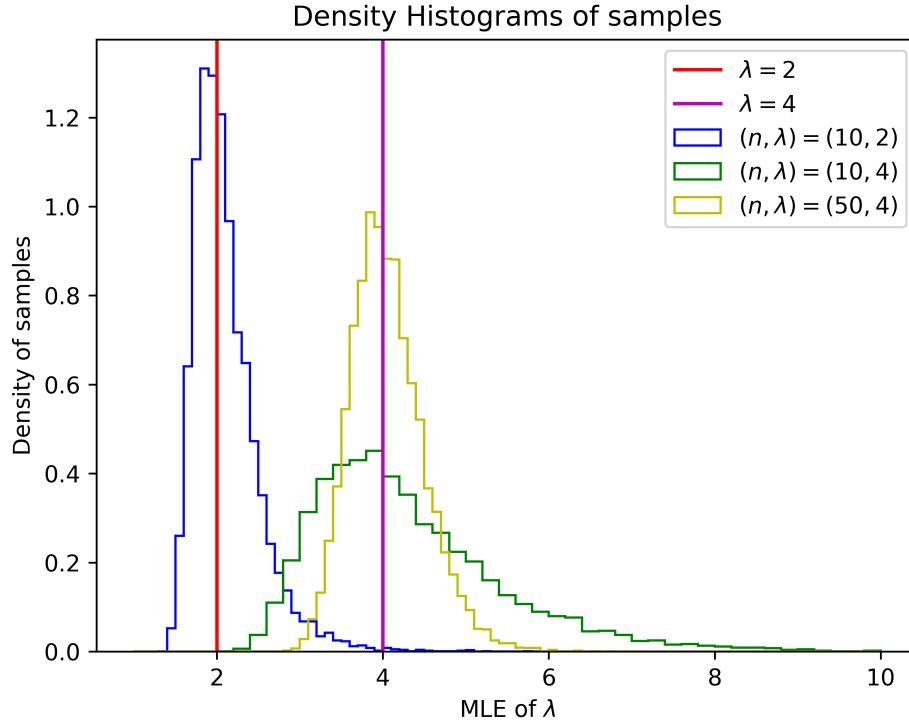
This can be done by using `numpy.random.rand()`.

Step 2: Apply the quantile function (with the parameter λ) to every sample from $U(0, 1)$.

Then we get the n samples of X when the true value of the parameter is λ .

Step 3: Compute $\hat{\lambda}$ using the formula from (b).

Step 4: Repeat this 10^4 times to get to get a sample $(\hat{\lambda}_1, \hat{\lambda}_2, \dots, \hat{\lambda}_{10^4})$. Then we create a density histogram from these 10^4 values.



(e)

From now on, we will refer to the array corresponding to (n, λ) as `mle_array_n_lambda`.
(And its sorted version will be referred as `sorted_mle_array_n_lambda`)

(i)

Our previous plot suggest that the rejection in this case should be a right-tailed one.

In order to compute a right-tailed **rejection region** of an `mle_array`, we use the following algorithm:

Step 1: Sort the MLE array.

This can be done using `numpy.sort()`.

We get the sorted array `sorted_mle_array`. Since the MLE array is now sorted, the empirical CDF is given by

$$\begin{aligned} ECDF(x) &= \frac{\text{number of elements } \leq x}{\text{length of the array}} && (\text{ECDF} = \text{empirical CDF}) \\ &= \frac{\text{index of } x \text{ in sorted_mle_array} + 1}{\text{length of the array}} && (\text{assuming that the list index starts with 0}) \end{aligned}$$

Step 2: Compute `index = ceil[ $\alpha \cdot \text{length}(\text{mle\_array})$ ] - 1`.

Then this will satisfy:

$$\text{ECDF}(\text{sorted_mle_array}[\text{index} - 1]) < \alpha \leq \text{ECDF}(\text{sorted_mle_array}[\text{index}])$$

$[\text{sorted_mle_array}[\text{index}]]$ will be the first element of the sorted_mle_array which is $\geq \alpha$

Step 3: The rejection region will be $(\text{sorted_mle_array}[\text{index}], +\infty)$.

In the given case, we obtain the following result:

Rejection region with Type I error 5%: $(2.83779124447156, +\infty)$

(ii)

In order to compute the **power** of the test, we use the following algorithm:

Step 1: Sort the mle_array_10.4.

This can be done using `numpy.sort()`.

We get the sorted array sorted_mle_array_10.4.

Step 2: The rejection region is of the form $(\text{rej}, +\infty)$. Find index such that:

$$\text{sorted_mle_array_10_4}[\text{index} - 1] < \text{rej} \leq \text{sorted_mle_array_10_4}[\text{index}]$$

This can be done by using `numpy.searchsorted()`.

Step 3: The power of the test is given by:

$$\text{power of the test} = \frac{\text{length of mle_array_10_4} - \text{index}}{\text{length of mle_array_10_4}}$$

[length of mle_array_10.4 – index is the number of the elements of the mle_array_10.4 that are in the rejection region]

In the given case, we obtain the following result:

The power of the test is: 0.9638

(iii)

In order to compute the **p-value** of an mle_observation (that is, MLE of an observation) given an mle_array (for which we assume that the rejection region is a right-tailed one), we use the following algorithm:

Step 1: Sort the MLE array.

This can be done using `numpy.sort()`.

We get the sorted array sorted_mle_array.

Step 2: Find index such that:

$$\text{sorted_mle_array}[\text{index} - 1] < \text{mle_observation} \leq \text{sorted_mle_array}[\text{index}]$$

This can be done by using `numpy.searchsorted()`.

Step 3: *The p -value is given by:*

$$p\text{-value} = \frac{\text{length of the mle_array} - \text{index}}{\text{length of the mle_array}}$$

[length of the mle_array – index is the number of the elements in mle_array which are $\geq$ mle_observation]

In the given case, we obtain the following result:

The MLE of this observation is: 2.478118666755411

Since the MLE is not in the rejection region, there is insufficient evidence to reject null hypothesis at the 5% level

The p -value of this observation is: 0.1419 (that is, 14.19%)